SHANGHAITECH·UNIVERSITY
2024-2025 SEMESTER 2 EXAM
SI100B INTRO TO INFORMATION SCIENCE AND TECHNOLOGY

April 18, 2025 Time Allowed: 90 minutes

NAME and **STUDENT ID:** 

Scores

Problem no.	1	2	3	4	5	Total

扫描全能王
3亿人都在用的扫描App

Exam Instructions

Please read and follow the instructions below carefully. Failure to follow the instructions may result in loss of points, which cannot be appealed.

1. Each problem requires you to write a function with a certain name. You must use the name specified in the problem, and not any other name. However, you can name the function parameters and variables anything you like.
2. Each problem specifies the number and types of input parameters (for example, a single list), and the type of the output (for example, a number). Your function must follow the specified input and output format.
3. The exam does not use an OJ. Therefore, you must test your programs properly to ensure they are correct.
4. Save your solutions to your local hard disk. Make a folder in the D:/ drive (not the Desktop directory), and name it "SID_SI100B_202402_exam", where SID is your student ID. E.g. if your student ID is 12345, name the folder "12345_SI100B_202402_exam". Save your solutions in this folder. For each problem, make a separate .py file with name "SID_pX.py", where SID is your student ID and X is the problem number, e.g. 12345_p3.py for your solution to problem 3. There are 5 problems, so you need to make 5 separate .py files.
5. Before you leave the exam, make sure all your files are saved as described above. If your files are not saved correctly, you may receive 0 points for certain problems.
6. Use only built-in Python functions in your code, and do not import any modules or packages. For some problems, you are also required to ***not*** use certain built-in Python functions.
7. The functions you write should be reasonably efficient. While you do not need to optimize your programs for maximum performance, extremely inefficient solutions may be marked down. For some problems, your program needs to terminate within certain specified time limits.
8. We placed monitors and chairs in specific positions. Do not move them. If you really need to move them for some reason, ask permission from the TA first.
9. Do not restart your computer unless absolutely necessary. If you need to restart, ask permission from the TA first. Note that restarting may cause you to ***lose data*** stored on the C:/ drive, but you will not lose data stored on the D:/ drive. This is why you should store your solutions on the D:/ drive.

扫描全能王
3亿人都在用的扫描App

1. Write a function called `count_inversions`, which takes as input a list of integers `L`, and returns the number of *inversions* in `L`. An inversion is a pair of numbers v and w from `L`, such that v occurs before w in `L`, but $v > w$.

   For example, given the input list [3,5,1,8,4], you should return 4, because there are 4 inversions, namely (3,1), (5,1), (5,4) and (8,4).

2. Write a function called `flatten`, which takes as input a list `L` in which every element is either an integer or a nested list of integers, and returns a list containing all the integers in `L` in left to right order. Note that `L` may be nested to an arbitrary depth.

   For example, given input [3,1,2,5] , you should return [3,1,2,5] . Given input [3, [1,5], [4, [6,1, [5]]], 2], you should return [3,1,5,4,6,1,5,2].

3. Write a function called `long_multiply`, which takes as input two lists `L1` and `L2`, each representing a (possibly very large) positive integer, and returns a list representing the product of the integers. Each element of `L1`, `L2` and the returned list is an integer between 0 to 9 representing one digit of the input or output values. Note that `L1` and `L2` may be quite large, and represent integers far exceeding the maximum value of Python's built-in `int` class. In other words, it is not possible to solve this problem by simply converting `L1` and `L2` to Python `int`'s and using the integer multiplication operator.

   For example, given input lists [2,1,3] and [9,7,0,8], you should return [2,0,6,7,8,0,4], because $213 \times 9708 = 2067804$.

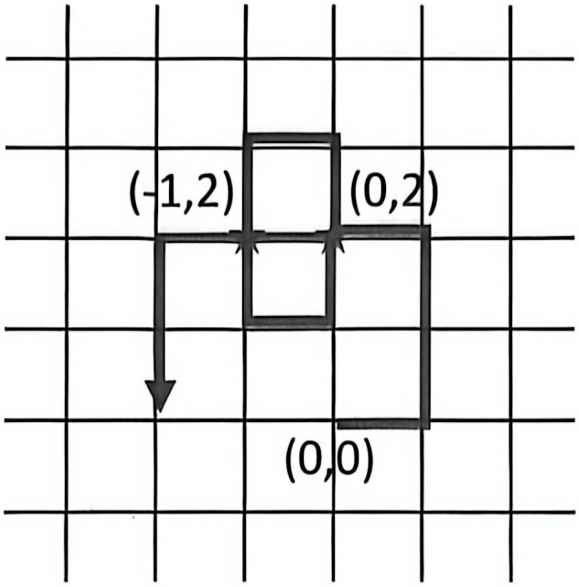
4. Write a function called `easy_sort`, which takes as input a list `L` containing integers, and returns a list with the values in `L` sorted in nondecreasing order. Each value in `L` is between 0 to 99, and some values may be repeated. Do not use Python functions related to sorting, such as `sorted()`, `sort()`, etc. in your solution. Instead, implement the sorting function by yourself. Your function should terminate in under 30 seconds for input lists containing up to 1 million elements.

   For example, given input [4,13,3,1,57,9,3], you should return [1,3,3,4,9,13,57].

5. Write a function called `crossing_path`, which takes as input a list `L` representing directions of travel of a path `P` on a grid, and outputs either the string "`No crossings`" if `P` never crosses itself, and otherwise outputs a list of 2-tuples containing all the locations at which `P` crosses itself, in order of the crossings. In particular, `P` starts at the origin (0,0). Each element of `L` is a string `'N'`, `'S'`, `'E'` or `'W'`, representing `P` moving one unit to the north, south, east or west. `P` crosses itself at a location (x, y) if (x, y) occurs in `P` more than once.

扫描全能王
3亿人都在用的扫描App

For example, given input `['N','E','E','S']`, this path which never crosses itself, so the output should be "No `crossings`". Given input `['E','N','N','W','N','W','S','S','E','N','W','W','S','S']`, this represents the path shown in the figure below. The output should be $[(0,2),(-1,2)]$, because `P` first crosses itself first at $(0,2)$, then at $(-1,2)$.



END OF EXAM

扫描全能王
3亿人都在用的扫描App